Syntax Directed Definition

Syntax Directed Definition (SDD) is a kind of abstract specification. It is generalization of context free grammar in which each grammar production X –> a is associated with it a set of production rules of the form s = f(b1, b2, ……bk) where s is the attribute obtained from function f. The attribute can be a string, number, type or a memory location. Semantic rules are fragments of code which are embedded usually at the end of production and enclosed in curly braces ({ }).

Example:

E --> E1 + T { E.val = E1.val + T.val}

Annotated Parse Tree – The parse tree containing the values of attributes at each node for given input string is called annotated or decorated parse tree.

Features – High level specification

Hides implementation details

Explicit order of evaluation is not specified

Types of attributes – There are two types of attributes:

1. Synthesized Attributes – These are those attributes which derive their values from their children nodes i.e. value of synthesized attribute at node is computed from the values of attributes at children nodes in parse tree. Example:

E --> E1 + T { E.val = E1.val + T.val}

In this, E.val derive its values from E1.val and T.val

Computation of Synthesized Attributes –

Write the SDD using appropriate semantic rules for each production in given grammar.

The annotated parse tree is generated and attribute values are computed in bottom up manner.

The value obtained at root node is the final output.

Example: Consider the following grammar

S --> E

E --> E1 + T

E --> T

T --> T1 * F

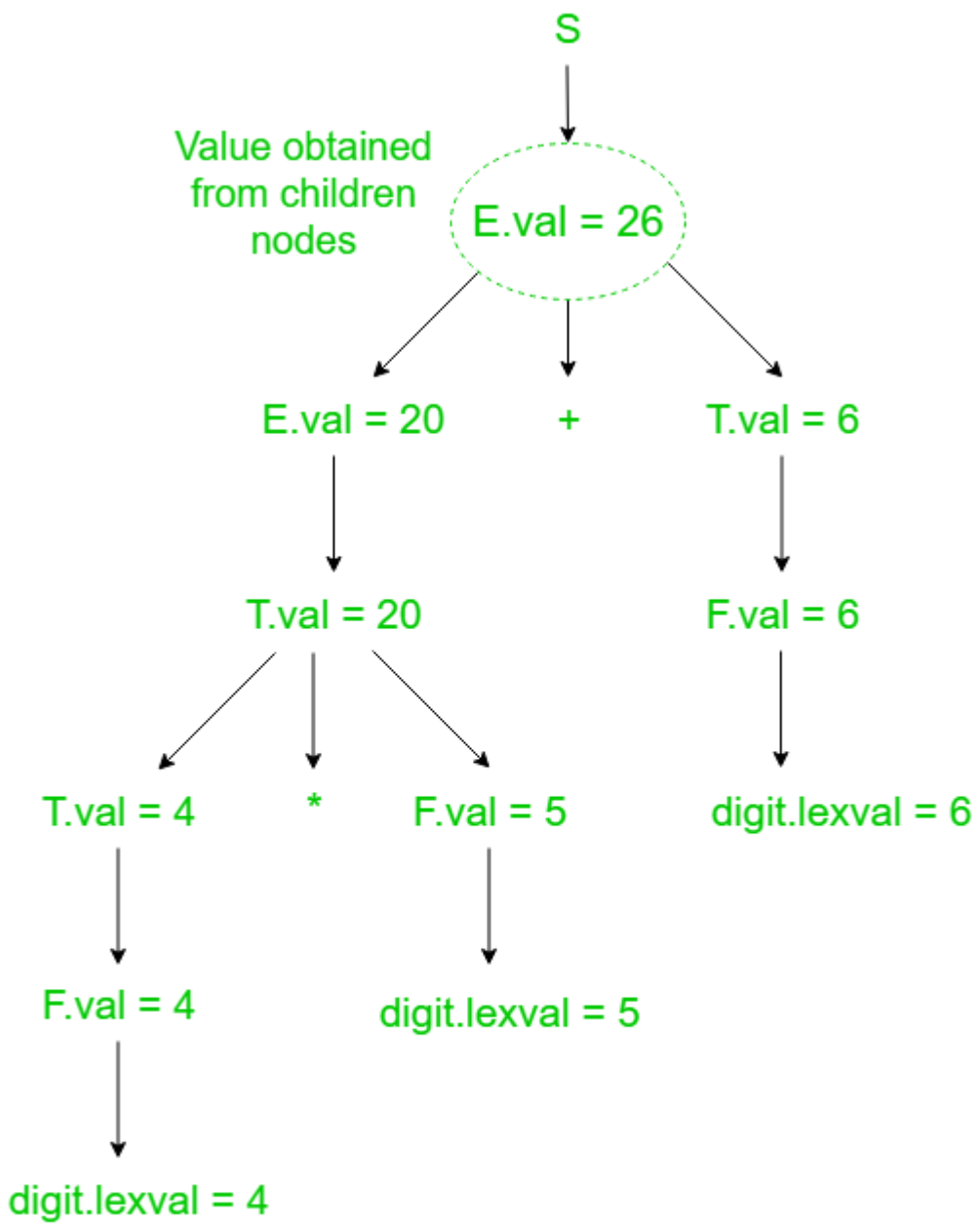
T --> F

F --> digit

The SDD for the above grammar can be written as follow

Production	Semantic Actions
S --> E	Print(E.val)
E --> E_1 + T	E.val = E_1.val + T.val
E --> T	E.val = T.val
T --> T_1 * F	T.val = T_1.val * F.val
T --> F	T.val = F.val
F --> digit	F.val = digit.lexval

Let us assume an input string 4 * 5 + 6 for computing synthesized attributes. The annotated parse tree for the input string is



Annotated Parse Tree

For computation of attributes we start from leftmost bottom node. The rule F –> digit is used to reduce digit to F and the value of digit is obtained from lexical analyzer which becomes value of F i.e. from semantic action F.val = digit.lexval. Hence, F.val = 4 and since T is parent node of F so, we get T.val = 4 from semantic action T.val = F.val. Then, for T –> T1 * F production, the corresponding semantic action is T.val = T1.val * F.val . Hence, T.val = 4 * 5 = 20

Similarly, combination of E1.val + T.val becomes E.val i.e. E.val = E1.val + T.val = 26. Then, the production S –> E is applied to reduce E.val = 26 and semantic action associated with it prints the result E.val . Hence, the output will be 26.

2. Inherited Attributes – These are the attributes which derive their values from their

parent or sibling nodes i.e. value of inherited attributes are computed by value of parent or sibling nodes.

Example:

A --> BCD { C.in = A.in, C.type = B.type }

Computation of Inherited Attributes –

Construct the SDD using semantic actions.

The annotated parse tree is generated and attribute values are computed in top down manner.

Example: Consider the following grammar

S --> T L

T --> int

T --> float

T --> double

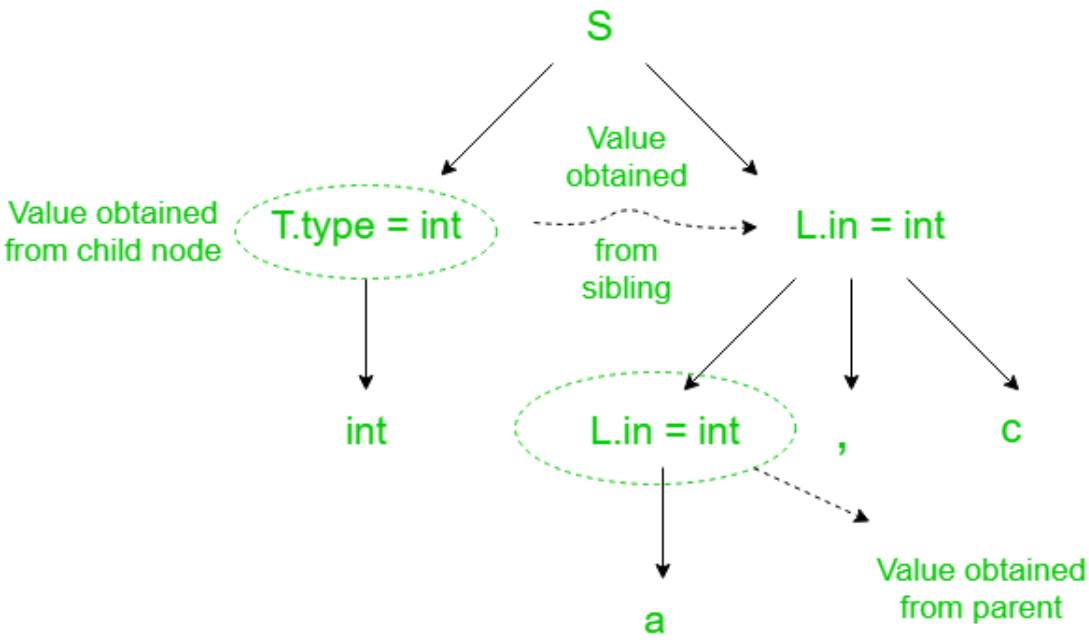
L --> L1, id

L --> id

The SDD for the above grammar can be written as follow

Production	Semantic Actions
S --> T L	L.in = T.type
T --> int	T.type = int
T --> float	T.type = float
T --> double	T.type = double
L --> L_1 , id	L_1.in = L.in Enter_type(id.entry , L.in)
L --> id	Entry_type(id.entry , L.in)

Let us assume an input string int a, c for computing inherited attributes. The annotated parse tree for the input string is



Annotated Parse Tree

The value of L nodes is obtained from T.type (sibling) which is basically lexical value obtained as int, float or double. Then L node gives type of identifiers a and c. The computation of type is done in top down manner or preorder traversal. Using function

Enter_type the type of identifiers a and c is inserted in symbol table at corresponding id.entry.

S – attributed and L – attributed SDTs in Syntax directed translation

Before coming up to S-attributed and L-attributed SDTs, here is a brief intro to Synthesized or Inherited attributes

Types of attributes –

Attributes may be of two types – Synthesized or Inherited.

1. Synthesized attributes –

A Synthesized attribute is an attribute of the non-terminal on the left-hand side of a production. Synthesized attributes represent information that is being passed up the parse tree. The attribute can take value only from its children (Variables in the RHS of the production).

For eg. let's say A -> BC is a production of a grammar, and A's attribute is dependent on B's attributes or C's attributes then it will be synthesized attribute.

2.Inherited attributes –

An attribute of a nonterminal on the right-hand side of a production is called an inherited attribute. The attribute can take value either from its parent or from its siblings (variables in the LHS or RHS of the production).

For example, let's say A -> BC is a production of a grammar and B's attribute is dependent on A's attributes or C's attributes then it will be inherited attribute.

Now, let's discuss about S-attributed and L-attributed SDT.

S-attributed SDT :

If an SDT uses only synthesized attributes, it is called as S-attributed SDT.

S-attributed SDTs are evaluated in bottom-up parsing, as the values of the parent nodes depend upon the values of the child nodes.

Semantic actions are placed in rightmost place of RHS.

L-attributed SDT:

If an SDT uses both synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from left siblings only, it is called as L-attributed SDT.

Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.

Semantic actions are placed anywhere in RHS.

For example,

A -> XYZ {Y.S = A.S, Y.S = X.S, Y.S = Z.S}

is not an L-attributed grammar since Y.S = A.S and Y.S = X.S are allowed but Y.S = Z.S violates the L-attributed SDT definition as attributed is inheriting the value from its right sibling.

L-attributed SDT:

If an SDT uses both synthesized attributes and inherited attributes with a restriction that inherited attribute can inherit values from left siblings only, it is called as L-attributed SDT.

Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.

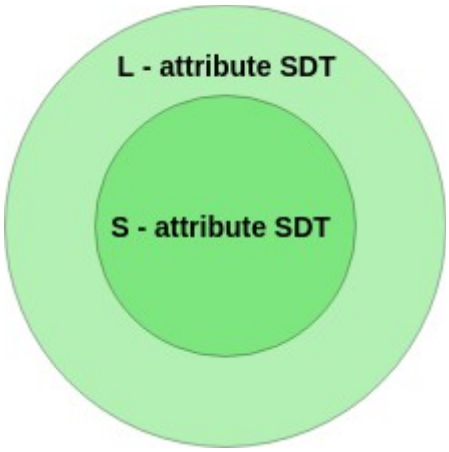
Semantic actions are placed anywhere in RHS.

For example,

A -> XYZ {Y.S = A.S, Y.S = X.S, Y.S = Z.S}

is not an L-attributed grammar since Y.S = A.S and Y.S = X.S are allowed but Y.S = Z.S violates the L-attributed SDT definition as attributed is inheriting the value from its right sibling.

Note – If a definition is S-attributed, then it is also L-attributed but NOT vice-versa.



Example – Consider the given below SDT.

P1: S -> MN {S.val= M.val + N.val}

P2: M -> PQ {M.val = P.val * Q.val and P.val =Q.val}

Select the correct option.

A. Both P1 and P2 are S attributed.

B. P1 is S attributed and P2 is L-attributed.

C. P1 is L attributed but P2 is not L-attributed.

D. None of the above

Explanation –

The correct answer is option C as, In P1, S is a synthesized attribute and in L-attribute definition synthesized is allowed. So P1 follows the L-attributed definition. But P2 doesn't follow L-attributed definition as P is depending on Q which is RHS to it.

Three address code in Compiler

Three address code is a type of intermediate code which is easy to generate and can be easily converted to machine code.It makes use of at most three addresses and one operator to represent an expression and the value computed at each instruction is stored in temporary variable generated by compiler. The compiler decides the order of operation given by three address code.

General representation –

a = b op c

Where a, b or c represents operands like names, constants or compiler generated temporaries and op represents the operator

Example-1: Convert the expression a * – (b + c) into three address code.

$$t_1 = b + c$$
$$t_2 = \text{uminus } t_1$$
$$t_3 = a * t_2$$

Example-2: Write three address code for following code

```
for(i = 1; i<=10; i++)
 {
  a[i] = x * 5;
 }
```

$$
\begin{aligned}
& i = 1 \\
L : \; & t_1 = x * 5 \\
& t_2 = \&a \\
& t_3 = sizeof(int) \\
& t_4 = t_3 * i \\
& t_5 = t_2 + t_4 \\
& *t_5 = t_1 \\
& i = i + 1 \\
& \text{if } i <= 10 \text{ goto } L
\end{aligned}
$$

Implementation of Three Address Code –

There are 3 representations of three address code namely

Quadruple

Triples

Indirect Triples

1. Quadruple –

It is structure with consist of 4 fields namely op, arg1, arg2 and result. op denotes the operator and arg1 and arg2 denotes the two operands and result is used to store the result of the expression.

Advantage –

Easy to rearrange code for global optimization.

One can quickly access value of temporary variables using symbol table.

Disadvantage –

Contain lot of temporaries.

Temporary variable creation increases time and space complexity.

Example – Consider expression a = b * – c + b * – c.

The three address code is:

t1 = uminus c

t2 = b * t1

t3 = uminus c

t4 = b * t3

t5 = t2 + t4

a = t5

#	Op	Arg1	Arg2	Result
(0)	uminus	c		t1
(1)	*	t1	b	t2
(2)	uminus	c		t3
(3)	*	t3	b	t4
(4)	+	t2	t4	t5
(5)	=	t5		a

Quadruple representation

2. Triples –

This representation doesn't make use of extra temporary variable to represent a single operation instead when a reference to another triple's value is needed, a pointer to that triple is used. So, it consist of only three fields namely op, arg1 and arg2.

Disadvantage –

Temporaries are implicit and difficult to rearrange code.

It is difficult to optimize because optimization involves moving intermediate code. When a triple is moved, any other triple referring to it must be updated also. With help of pointer one can directly access symbol table entry.

Example – Consider expression a = b * – c + b * – c

#	Op	Arg1	Arg2
(0)	uminus	c	
(1)	*	(0)	b
(2)	uminus	c	
(3)	*	(2)	b
(4)	+	(1)	(3)
(5)	=	a	(4)

Triples representation

3. Indirect Triples –

This representation makes use of pointer to the listing of all references to computations which is made separately and stored. Its similar in utility as compared to quadruple representation but requires less space than it. Temporaries are implicit and easier to rearrange code.

Example – Consider expression a = b * – c + b * – c

#	Op	Arg1	Arg2
(14)	uminus	c	
(15)	*	(14)	b
(16)	uminus	c	
(17)	*	(16)	b
(18)	+	(15)	(17)
(19)	=	a	(18)

List of pointers to table

#	Statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

Indirect Triples representation

Question – Write quadruple, triples and indirect triples for following expression : (x + y) * (y + z) + (x + y + z)

Explanation – The three address code is:

t1 = x + y

t2 = y + z

t3 = t1 * t2

t4 = t1 + z

t5 = t3 + t4

#	Op	Arg1	Arg2	Result
(1)	+	x	y	t1
(2)	+	y	z	t2
(3)	*	t1	t2	t3
(4)	+	t1	z	t4
(5)	+	t3	t4	t5

Quadruple representation

#	Op	Arg1	Arg2
(1)	+	x	y
(2)	+	y	z
(3)	*	(1)	(2)
(4)	+	(1)	z
(5)	+	(3)	(4)

Triples representation

#	Op	Arg1	Arg2
(14)	+	x	y
(15)	+	y	z
(16)	*	(14)	(15)
(17)	+	(14)	z
(18)	+	(16)	(17)

List of pointers to table

#	Statement
(1)	(14)
(2)	(15)
(3)	(16)
(4)	(17)
(5)	(18)

Indirect Triples representation

Detection of a Loop in Three Address Code

Loop optimization is the phase after the Intermediate Code Generation. The main intention of this phase is to reduce the number of lines in a program. In any program majority of the time is spent by any program is actually inside the loop for an iterative program. In the case of the recursive program a block will be there and the majority of the time will present inside the block.

Loop Optimization –

1.To apply loop optimization we must first detect loops.

2.For detecting loops we use Control Flow Analysis(CFA) using Program Flow Graph(PFG).

3.To find program flow graph we need to find Basic Block

Basic Block – A basic block is a sequence of three address statements where control enters at the beginning and leaves only at the end without any jumps or halts.

Finding the Basic Block –

In order to find the basic blocks, we need to find the leaders in the program. Then a basic block will start from one leader to the next leader but not include the next leader. This means if you find out that line no 1 is a leader and line no 15 is the next leader, then the line from 1 to 14 is a basic block, but not including line no 15.

Identifying leader in a Basic Block –

First statement is always a leader

Statement that is target of conditional or un-conditional statement is a leader

Statement that follows immediately a conditional or un-conditional statement is a leader

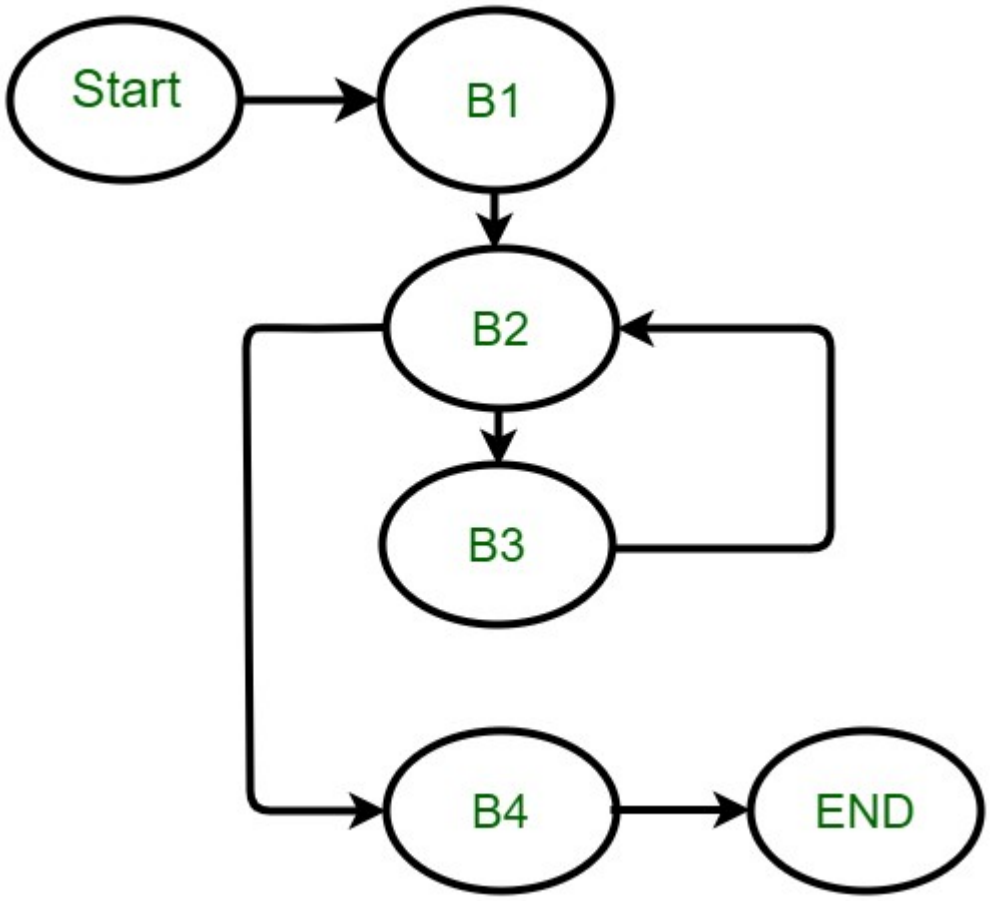
```
fact(x)
{
   int f = 1;
   for (i = 2; i <= x; i++)
      f = f * i;
   return f;
}
```

Three Address Code of the above C code:

```
1.f = 1;
2 i = 2;
3if (i > x) goto 9
4 t1 = f * i;
5 f = t1;
6 t2 = i + 1;
7 i = t2;
8 goto(3)
9 goto calling program
```

Leader and Basic Block –

```
1.f = 1;*
2.i = 2;          B1
3.if (i > x) goto 9*   B2
4.t1 = f * i;*
5.f = t1;
6.t2 = i + 1;     B3
7.i = t2;
8.goto(3)
9.goto calling program*   B4
```



If the control enter B1 there is no other option after B1, it has to enter B2. Now, if control

enters B2, then depending on the condition control will flow, if the condition is true we are going to line no 9, which means 9 is nothing but B4. But if the condition is false control goes to the next block B3. After B3, there is no condition at all we are direct goes to 3rd statement B2. Above control flow graph have a cycle between B2 and B3 which is nothing but a loop.